

Authentication and Access Control

[Q-1] Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

- **Python Code** username = input

("Enter Username: ") password =

input ("Enter Password: ")

admin_username = "admin" admin_password

= "zomato123"

if username == admin_username and password == admin_password:

print ("Access Granted")

else:

print ("Access Denied")

- **Sample Output** Enter Username:

admin

Enter Password: zomato123

Access Granted

[Q-2] Write a Python function check access (user role, resource) that returns True if the user role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'Restaurant owner', and resources like 'menu edit', 'order history', 'admin_dashboard'. Hint: Use a dictionary to map roles to allowed resources.

- **Python Code**

```
def check access (user role, resource):  
    permissions = {  
        "customer": ["order history"],  
        "Restaurant owner": ["menu edit", "order history"],  
        "admin": ["menu edit", "order history", "admin_dashboard"]  
    }  
  
    return resource in permissions. Get (user role, [])  
  
# Test Cases print (check access ("customer", "order  
history")) print (check access ("customer", "menu edit"))  
print (check access ("restaurant owner", "menu edit")) print  
(check access ("restaurant owner", "admin_dashboard"))  
print (check access ("admin", "admin_dashboard"))
```

- **Output**

True
False
True
False
True

[Q-3] Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

- **Python Code**

```
import random
correct_password = "paytm123"
password = input("Enter Password: ")
if password == correct_password:
    otp = random.randint(100000, 999999)
    print("Generated OTP:", otp)
    user_otp = int(input("Enter OTP: "))
    if user_otp == otp:
        print("Login Successful")
    else:
        print("Invalid OTP")
else:
    print("Incorrect Password")
```

- **Sample Output**

Enter Password: paytm123

Generated OTP: 456789

Enter OTP: 456789

Login Successful

**[Q-4] Given an array of user objects with properties: username, is Premium, and last Login, write a function that returns only those users who are premium and have logged in within the last 7 days.

Constraint: Assume last Login is a date string in 'YYYYMM-DD' format, and today is '2024-06-10'.**

- **Python Code**

```
from datetime import datetime, time delta
```

```
def get_active_premium_users(users):
```

```
today = datetime.strptime("2024-06-10", "%Y-%m-%d")
seven_days_ago = today - time delta(days=7)

result = [] for
user in users:

    login date = datetime.strptime(user["lastLogin"], "%Y-%m-%d")
if user["is Premium"] and login date >= seven_days_ago:

    result.
Append(user) return
result users = [

    {"username": "harsh", "is Premium": True, "last Login": "2024-06-08"},
    {"username": "Rahul", "is Premium": False, "last Login": "2024-06-09"},
    {"username": "Priya", "is Premium": True, "last Login": "2024-05-30"},
    {"username": "Neha", "is Premium": True, "last Login": "2024-06-05"}

]
print(get_active_premium_users(users))
```

• Output

```
{'username': 'harsh', 'is Premium': True, 'last Login': '2024-06-08'},
{'username': 'Neha', 'is Premium': True, 'last Login': '2024-06-05'}
```

[Q-5] Use ChatGPT to generate a list of 5 common access control mistakes developers make in real-world apps like Instagram or Flipkart, and for each, write a one-line fix or prevention tip.

Mistake	Fix / Prevention
1. Using weak passwords	Enforce strong password policies and complexity rules.

2. Not implementing role-based permissions	Use RBAC (Role-Based Access Control) for every resource.
3. Exposing admin pages through direct URLs	Verify user authorization on every request.
4. Storing passwords in plain text	Store passwords using secure hashing algorithms like crypt.
5. Not enabling multi-factor Authentication (MFA)	Require OTP or authenticator app verification for sensitive actions.